Scalable OTT Recommendation and Churn-Signal Detection with Apache Spark

[Author 1 Name]
Dept. of Computer Engineering
Vidyalankar Institute of Technology, Mumbai, India
email1@vit.edu.in

[Author 2 Name]
Dept. of Computer Engineering
Vidyalankar Institute of Technology, Mumbai, India
email2@vit.edu.in

***Abstract*—Over-the-top (OTT) streaming platforms depend on recommendation engines to keep viewers engaged, and on early detection of disengagement to reduce subscriber churn. Both tasks operate on very large, sparse and continuously growing interaction logs. We present an end-to-end Big Data pipeline built on Apache Spark that (i) ingests the MovieLens 25M ratings corpus into columnar storage, (ii) trains a distributed Alternating Least Squares (ALS) collaborative-filtering model with a leak-free chronological split, (iii) derives an engagement-decay churn signal from behavioural features, and (iv) demonstrates low-latency serving with Spark Structured Streaming. The tuned ALS model attains a test RMSE of – against – for a global-mean baseline, and a Precision@10 of – versus – for a popularity baseline. A gradient-boosted churn classifier reaches an ROC-AUC of –. All code and instructions are publicly available.**

***Index Terms*—Big Data, Apache Spark, OTT analytics, collaborative filtering, ALS, churn prediction**

I. Introduction

Subscription video platforms compete on retention. Industry accounts of large OTT services report that personalised recommendation is central to how viewers find content and therefore to the value of the service [7]. A viewer who cannot quickly find something to watch is more likely to disengage, and sustained disengagement is the strongest precursor of cancellation.

The underlying data is a textbook Big Data workload. Interaction logs reach billions of events; the user–item matrix is extremely sparse; new events arrive continuously; and the quality of the signal is uneven because popular titles dominate. Single-node tools struggle with the joins, shuffles and iterative factorisation that these workloads require, which motivates distributed engines such as Apache Spark [2], [3].

Contributions. (1) A reproducible Spark pipeline from raw CSV to ranked recommendations, with all stages scripted. (2) A rigorous evaluation protocol that uses a chronological split, a rating-and-ranking metric suite and catalogue coverage. (3) An engagement-decay churn signal computed strictly from pre-cutoff behaviour. (4) A streaming serving demonstration that illustrates the velocity dimension of the problem.

II. Related Work

Matrix factorisation is the dominant family of collaborative-filtering methods since the Netflix Prize [4]. For implicit and explicit feedback, Hu et al. introduced a confidence-weighted least-squares formulation [5], and Zhou et al. showed that alternating least squares parallelises naturally across machines [6], which is the basis of Spark MLlib's implementation [3]. Gradient boosting [9] is a standard strong baseline for tabular behavioural prediction such as churn. Spark Structured Streaming [8] extends the same DataFrame API to unbounded data. We combine these established components into a single OTT-oriented pipeline and focus on evaluation hygiene (time-aware splitting, cold-start handling and ranking metrics) rather than on a new algorithm.

III. Dataset and Problem Formulation

We use the MovieLens 25M dataset [1], which contains – ratings from – users on – movies, each with a 0.5–5.0 star score and a timestamp. We use it as a public proxy for OTT viewing feedback.

Task A (recommendation). Given a user's history up to time t, rank unseen titles. Predicted ratings are scored by RMSE; ranked lists by Precision@K and Recall@K with K=10, treating ratings ≥ 4 as relevant.

Task B (churn signal). Let t_c be the 80th percentile of event time. A user with at least five events before t_c is labelled *churned* if they produce no events after t_c. This is a disengagement proxy, because the dataset has no subscription cancellations; it should be read as an engagement signal and not as true billing churn.

A. Evaluation metrics

For a test set T of ratings, $\mathrm{RMSE} = \sqrt{|T|^{-1}\sum_{(u,i)\in T}(r_{ui}-\hat{r}_{ui})^2}$. For user u with ranked list L_u^K and relevant set $\mathcal{R}_u$,

$$\mathrm{P@}K = \frac{|L_u^K \cap \mathcal{R}_u|}{K}, \quad \mathrm{R@}K = \frac{|L_u^K \cap \mathcal{R}_u|}{|\mathcal{R}_u|}, \qquad (1)$$

averaged over users. Catalogue coverage is $|\bigcup_u L_u^K|/|\mathcal{I}|$, where $\mathcal{I}$ is the set of titles. For churn we report ROC-AUC and PR-AUC, because the positive class can be imbalanced.

B. Big Data characteristics

IV. System Architecture

Fig. 1 shows the pipeline. Each stage is a separate Spark job that reads and writes Parquet, so stages can be rerun independently.

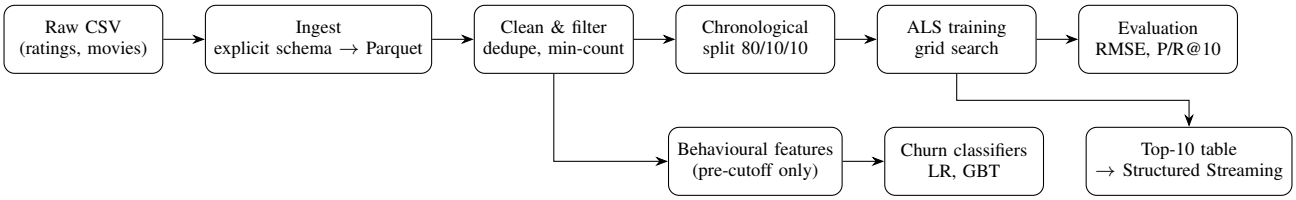


Fig. 1. End-to-end pipeline. Every arrow is a Parquet hand-off between independent Spark jobs.

TABLE I
MAPPING OF THE FIVE VS TO THIS OTT WORKLOAD.

Characteristic	In this project
Volume	Millions of ratings; user–item matrix with $\gg$99% missing entries
Velocity	Continuous viewing events; simulated via Structured Streaming
Variety	Ratings, timestamps, multi-valued genre strings
Veracity	Popularity bias, self-selected raters, sparse users/items
Value	Better discovery and earlier detection of disengagement

A. Ingestion and cleaning

CSVs are read with explicit schemas, which avoids a schema-inference pass over the data, then de-duplicated on $(user, item)$ and written as Parquet. Ingestion took – s on our machine. Users and items with fewer than five ratings are removed to limit extreme sparsity; remaining cold-start cases in the test period are handled in Section V.

B. Chronological split

A random split would let the model train on events that happen after test events. We instead split on global timestamp quantiles: the earliest 80% for training (– rows), the next 10% for validation (–) and the final 10% for testing (–).

C. ALS model

ALS approximates the rating matrix $R \in \mathbb{R}^{m\times n}$ as $R \approx UV^\top$ with $U \in \mathbb{R}^{m\times k}$, $V \in \mathbb{R}^{n\times k}$, minimising

$$\sum_{(u,i)\in\Omega} \big(r_{ui} - \mathbf{u}_u^\top \mathbf{v}_i\big)^2 + \lambda\Big(\sum_u \|\mathbf{u}_u\|^2 + \sum_i \|\mathbf{v}_i\|^2\Big). \quad (2)$$

Fixing V makes the problem a set of independent per-user least-squares solves, and vice versa, so each half-step is embarrassingly parallel across partitions [3], [6]. The core of the implementation is shown below.

```
als = ALS(userCol="userId", itemCol="movieId",
       ratingCol="rating", rank=k, regParam=lam,
       maxIter=10, nonnegative=True,
       coldStartStrategy="drop", seed=42)
model = als.fit(train_plus_val)
recs = model.recommendForUserSubset(users, 200)
```

We grid-search rank k and regularisation λ on the validation split, then retrain on train+validation with the best setting and score the held-out test period. We use non-negative factors and `coldStartStrategy=drop`.

TABLE II
BEHAVIOURAL FEATURES USED FOR CHURN PREDICTION (ALL COMPUTED BEFORE t_c).

Feature	Meaning
`n_ratings`, `n_titles`	Event count; distinct titles
`avg_rating`, `std_rating`	Mean and spread of scores
`n_genres`	Genre diversity
`span_days`	First-to-last event interval
`days_since_last`	Recency of last event
`r_last30`, `r_last90`	Events in last 30/90 days
`recent_ratio`	`r_last90`/`n_ratings`

D. Churn features and models

Features are computed strictly from events before t_c: rating count, mean and standard deviation of ratings, distinct titles, genre diversity, activity span, days since last event, and counts in the last 30 and 90 days (with their ratio to total activity). We compare logistic regression with a gradient-boosted tree classifier [9] on a random 80/20 user split.

E. Streaming serving

The final top-10 lists are stored as a table. A rate-source stream of synthetic viewing events is joined with that table inside Spark Structured Streaming [8] with 2-second micro-batches, simulating on-demand recommendation lookups. In a 15-second test it served – lists. This demonstrates the serving pattern rather than production latency.

V. EXPERIMENTAL SETUP

All jobs ran in Spark local mode with PySpark. Hyper-parameters were rank $\in \{10, 20, 50\}$ and $\lambda \in \{0.05, 0.1, 0.2\}$ with 10 iterations. **Cold start:** test rows whose user or item is absent from training are dropped by ALS; we report the scored share as coverage (–%). **Ranking evaluation:** for a random sample of – test users with at least one relevant item, we request 200 candidates from ALS, remove titles the user has already rated in train+validation, and keep the top 10. The popularity baseline recommends the most-rated unseen titles. We also report *catalogue coverage*, the share of all titles that appear in any recommended list, as a proxy for diversity.

Reproducibility. The repository provides one script per stage and a wrapper (`run_all.sh`) that downloads the data, runs every stage, and regenerates the numeric macros and figure used in this paper, so each table entry traces to a JSON file under `results/`.

TABLE III
EXPERIMENTAL ENVIRONMENT AND CONFIGURATION.

Item	Value
Hardware	[CPU model, cores, RAM]
Software	Spark 4.x (PySpark), Python 3.12
Mode / driver memory	local[*] / [e.g. 8 GB]
ALS grid	rank $\{10, 20, 50\}$, $\lambda\{0.05, 0.1, 0.2\}$
ALS iterations / seed	10 / 42
Shuffle partitions	64
Relevance threshold	rating ≥ 4.0

TABLE IV
RATING PREDICTION ON THE CHRONOLOGICAL TEST SET (LOWER IS BETTER).

Model	RMSE
Global mean	–
Item mean	–
ALS (rank=–, λ=–)	–

VI. RESULTS

A. Rating prediction

Table IV compares ALS with two non-personalised baselines. The grid-search validation curve is shown in Fig. 2. The final ALS fit on train+validation took – s.

B. Top-10 ranking quality

Table V reports ranking metrics. Because relevance is defined by the future test window and already-seen titles are excluded, absolute precision values are low by construction; the relative comparison between methods is what matters. Catalogue coverage indicates how much of the catalogue each method surfaces; popularity concentrates on a small head of titles.

C. Churn signal

Table VI lists classifier performance. The most important GBT features were ––, –– and ––.

VII. DISCUSSION

Scalability. Parquet storage with explicit schemas and a single shared Spark session per stage keep ingestion and feature computation inexpensive, and the ALS half-steps parallelise across partitions. Moving from local mode to a cluster requires only changing the master URL and partition counts.

Threats to validity. (i) Ratings are explicit feedback, whereas real platforms mostly observe implicit signals such as watch time. (ii) The churn label is an activity proxy, not a billing event. (iii) MovieLens users chose to rate, so selection bias exists. (iv) The streaming demo uses synthetic events. (v) Results come from a single run with a fixed seed; we did not compute confidence intervals.

Complexity. Each ALS iteration performs two sets of $k \times k$ normal-equation solves, costing roughly $O\big(|\Omega|k^2 + (m+n)k^3\big)$ per iteration, plus shuffle traffic proportional to $|\Omega|k$. Rank therefore trades accuracy against both compute and network cost, which is why the grid search records fit time alongside validation RMSE.

Figure is generated by `make_paper_assets.py` after the experiments are run.

Fig. 2. Validation RMSE versus latent rank for each regularisation value.

TABLE V
TOP-10 RECOMMENDATION QUALITY ON – SAMPLED TEST USERS.

Model	P@10	R@10	Catalogue cov. (%)
Popularity	–	–	–
ALS	–	–	–

Cold start and fairness of evaluation. Dropping unseen users and titles at test time flatters RMSE slightly because the hardest rows are excluded; we therefore report the scored share. A production system would route such rows to a content-based or popularity fallback.

Practical implications. A recommender is only useful if it exposes more of the catalogue than a popularity list while remaining accurate; reporting coverage next to accuracy makes that trade-off visible. The churn features are cheap to refresh and can feed retention actions such as personalised rows or re-engagement messages.

VIII. CONCLUSION AND FUTURE WORK

We built and evaluated an open Spark pipeline for OTT recommendation and engagement-decay detection. Future work includes implicit-feedback ALS on watch-time data, a two-tower neural retrieval model, a feature store with Kafka ingestion for true streaming, and evaluation on data with real subscription outcomes.

Ethical considerations. Recommenders can reinforce popularity bias and narrow what users see. Reporting catalogue coverage is one safeguard; periodic audits of exposure by genre and release year are another. MovieLens contains no personal identifiers beyond anonymised user IDs, and no scraping or private data were used.

CONTRIBUTION STATEMENT

[Author 1]: data ingestion, cleaning, chronological split, churn features, repository and README. **[Author 2]:** ALS modelling, hyper-parameter search, evaluation metrics, streaming demo and figures. Both authors designed the architecture and wrote and reviewed the paper.

CODE AVAILABILITY

Code, run scripts and instructions: https://github.com/[username]/ott-bigdata-pipeline.

REFERENCES

[1] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, pp. 1–19, 2015.

[2] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[3] X. Meng *et al.*, "MLlib: Machine learning in Apache Spark," *J. Mach. Learn. Res.*, vol. 17, no. 34, pp. 1–7, 2016.

TABLE VI
DISENGAGEMENT PREDICTION ON HELD-OUT USERS (– USERS, –% LABELLED CHURNED).

Model	ROC-AUC	PR-AUC
Logistic regression	–	–
Gradient-boosted trees	–	–

[4] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

[5] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *Proc. IEEE ICDM*, 2008, pp. 263–272.

[6] Y. Zhou, D. Wilkinson, R. Schreiber, and R. Pan, "Large-scale parallel collaborative filtering for the Netflix Prize," in *Proc. AAIM*, 2008, pp. 337–348.

[7] C. A. Gomez-Uribe and N. Hunt, "The Netflix recommender system: Algorithms, business value, and innovation," *ACM Trans. Manage. Inf. Syst.*, vol. 6, no. 4, pp. 1–19, 2015.

[8] M. Armbrust *et al.*, "Structured Streaming: A declarative API for real-time applications in Apache Spark," in *Proc. ACM SIGMOD*, 2018, pp. 601–613.

[9] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.